

1CrTrader — Product Features Document

Version: 1.0 (Prototype) | **Audience:** Users, Stakeholders, Developers | **Status:** Frontend prototype, no live backend or real broker connection yet

What This Application Is All About

1CrTrader is a discipline-enforcement app for Indian retail traders in futures and options (F&O) — not a signals service, not a tip-sharing platform, and deliberately so. The product's founding insight is that most retail traders don't lose money because they lack good setups; they lose money because they ignore their own rules under pressure. Random entries, revenge trades after a loss, holding a position overnight out of hope rather than plan, oversized "zero hero" bets on expiry day — these behavioral failures, not bad analysis, are what erode a trading account over time. 1CrTrader's job is to sit between the trader and their broker and make discipline the path of least resistance: it computes exactly how much capital a trader is allowed to risk today based on their account size, caps lot sizes and daily loss before a bad day becomes a catastrophic one, and enforces cooldowns between trades so a losing streak can't be chased in real time.

The target user is a self-directed retail F&O trader — someone trading their own capital through a discount broker like Zerodha, typically with account sizes from ₹25,000 up toward ₹1 crore (the name "1Cr" is literally the milestone the app is built around: helping a trader compound their way to a crore without blowing up along the way). The app is currently a frontend-only prototype — every screen, calculation, and rule described in this document is fully built and interactive, but data is not yet persisted to a real backend, and broker connections are simulated rather than live. This document describes the product as it exists today, feature by feature, and is written to be equally useful to someone deciding whether to use the app, someone deciding whether to fund the next phase of its development, and someone who has to pick up the codebase and extend it.

Set Up Your Profile (Onboarding Wizard)

The Big Picture (For Normal Users)

Before you can use 1CrTrader, it needs to know who you are as a trader — not your name, but your actual trading reality: how much capital you're working with, what kind of trader you are (do you buy options, sell them, trade futures?), and which instruments you trade. You can either connect your broker account and let the app pull this in automatically, or set it up by hand in a few quick steps. Either way, by the end you'll see exactly what risk rules apply to you specifically, not generic advice.

The Business Value (For Stakeholders)

This onboarding flow is the single highest-leverage screen in the product — it's the difference between a trader who understands and trusts the rules being enforced on them, and one who feels arbitrarily restricted and churns. The broker-connect path also doubles as the foundation for the product's eventual real differentiator: live, automatic enforcement, rather than a static calculator a trader has to remember to check. Every minute spent making this flow frictionless directly reduces drop-off before a trader ever experiences the app's core value.

The Blueprint (For Upcoming Developers)

Implemented as a 4-step wizard (Broker → Trading Style → Instruments → Confirm), built deliberately to diverge from a naive linear wizard because the underlying state doesn't split cleanly into 4 independent steps:

- **Step 1 (Broker or Manual):** Clicking a broker tile triggers a simulated async connect (`connectBroker()`, ~1.4s delay) that sets `selectedTier`, `startingCapital`, `currentBalance`, and the instrument list all atomically, since a real broker API would return all of this in one response. "Set up manually instead" swaps in a tier-card grid + capital input in the same step — confirmed deliberately not to live on Step 3, since tier and capital are one decision, not two.
- **Step 3 (Instruments):** Shows the broker-fetched list (pre-selected) or a manual searchable/category picker, depending on which path Step 1 took.
- **Step 4 (Confirm):** Always read-only — this was a deliberate fix; an earlier version asked the trader to re-type their already-fetched balance, which made no sense once a broker had already supplied it. No "suggested tier" shown here, since tier is a pure function of capital, not of trading style.
- State lives in module-scoped JS variables in `dashboard.js` (`selectedTier`, `startingCapital`, `selectedTraderTypes` as a `Set`, `selectedInstruments` as a keyed object) — **not yet persisted**; a page reload mid-onboarding loses progress. This is the most consequential gap for a backend integration to close first.
- **Edge cases already handled:** capital amount outside the selected tier's valid range (inline validation against `TIER_RANGES`), re-opening this screen via "Change Tier" from Account (detects an existing profile and skips to the manual-edit state rather than re-prompting for a broker).
- **Edge cases NOT yet handled:** network failure during a real broker OAuth handshake (currently impossible to simulate, since the connect is a local `setTimeout`); partial profile state surviving a crash mid-wizard.

Capital Tier & Risk Rules Engine

The Big Picture (For Normal Users)

This is the invisible math running behind almost everything else in the app. Based on how much money you're trading with, the app tells you exactly how much you're allowed to lose in a single day, and how many lots you can trade — and these numbers get a little more generous as your account grows, but never relax on a day when you've just lost money. A trader with ₹25,000 can lose up to 7% of it in a day; a trader with ₹19,00,000+ is capped at a flat 2%. The bigger your account, the smaller the percentage you're allowed to risk — because in absolute rupees, 2% of a large account is already a serious amount of money.

The Business Value (For Stakeholders)

This tiered-percentage model is the product's core IP and the reason a trader would choose 1CrTrader over a generic spreadsheet — it encodes a specific, defensible risk philosophy (smaller accounts can tolerate a higher loss percentage because the absolute rupee amount is still survivable; larger accounts cannot). It is also the foundation every other feature in the app reads from, which means it needs to be exactly right and exactly consistent — a discrepancy here (e.g., two different parts of the app disagreeing on a trader's allowed loss) directly undermines the product's entire value proposition of being a trustworthy guardrail.

The Blueprint (For Upcoming Developers)

Single source of truth: `tierRulesMatrix` in `dashboard.js`, a flat object keyed `"small-1"` through `"pro-3"` (12 sub-tiers across 4 broad tiers — Small/Medium/Large/Pro), each entry holding `{ cap, pct, loss, maxLots }`.

- **Two independent axes determine a trader's actual sub-tier**, both read through one function, `getOfficialSubLevelKey()`, so lot count and loss amount can never silently disagree:
 1. **Entry sub-tier** — whichever sub-tier the trader's *original* starting capital falls into. Depositing ₹60,000 into the Small tier enters directly at small-2, not small-1.
 2. **Growth unlock** — every +50% profit on *original* starting capital (compounding: 1.5×, 2.25×, ...) advances one sub-tier.
- **Ratchet mechanic:** `highestOfficialSubLevelNum`/`highestOfficialSubLevelTier` track the highest sub-tier ever reached — a losing day can reduce current balance, but never un-earns a previously unlocked lot/loss tier. This is a one-way function by design.
- **Max loss never multiplies by lots** — `getCurrentMaxLossRupees()` returns the sub-tier's flat `loss` figure regardless of whether the trader uses 1 lot or their full allowance. A separate function, `getPerLotMaxLossRupees(lotsUsed)`, computes `((tier.loss / tier.maxLots) * lotsUsed)` for screens that need to show what a *partial* lot allocation is actually allowed to risk (see Daily Limits Tool below) — these are deliberately different numbers for different questions, and conflating them was a real bug caught and fixed during development.
- **Edge cases handled:** a trader whose balance drops back below a previous unlock threshold after a loss still keeps their unlocked lot count (ratchet); manual tier

selection validates the entered capital against `TIER_RANGES` before accepting it.

- **Known gap:** all 12 sub-tier figures are hardcoded constants. A real backend would need this matrix to be configurable (e.g., for regulatory changes to leverage rules) without a code deploy.
-

Daily Limits Tool

The Big Picture (For Normal Users)

This is your main working screen — where you actually log today's trades and watch your risk rules in real time. It shows you, instrument by instrument, exactly how many points of stop-loss you're allowed before you'd breach your daily limit, and lets you type in your own planned stop-loss to see what it would actually cost in rupees — with a clear warning if that number is bigger than what you're allowed for the lots you're using. There's also a full reference table showing every tier's rules side by side, in case you want to understand why a smaller account is allowed a bigger loss percentage than a larger one — tucked away collapsed by default so it doesn't clutter the page, but always one click away.

The Business Value (For Stakeholders)

This screen is where daily retention lives — it's the one a trader needs to open every single trading day, which makes it the product's natural habit-forming surface. The per-instrument, per-lot breakdown (rather than just a single daily number) is a meaningful differentiator: it answers the question a trader actually has in the moment ("if I take this specific trade, am I within my rules?") rather than making them do the math themselves.

The Blueprint (For Upcoming Developers)

Two merged sections, deliberately combined from what were originally two separate tables after user feedback that two tables side-by-side was confusing:

- **Summary chips** (`instrument-tier-summary`) showing the trader's own tier/max-lots/max-loss once, rather than repeating it on every row.
- **Live instrument table**, one row per selected instrument, columns: Qty/Lot (editable for stocks via `onSlTableQtyInput`, locked for indices since exchange-set lot sizes change rarely), Lots (editable, clamped to `getMaxAllowedLots()`), Max Lots Allowed (read-only reference), Points SL (`= perLotBudget / (qty × lots)`), NOT the flat tier loss — this was a real, fixed bug where 1 lot was incorrectly shown the *full* tier budget instead of its proportional share), Your SL (pts) (free input), Loss If Hit (Rs.) (`= typed SL × qty × lots`), stays empty until the trader actually types a value — an earlier version defaulted this to the Points SL figure and was reverted after user feedback that it read as cluttered).
- **Over-allowance warning:** when `Loss If Hit > getPerLotMaxLossRupees(lotsUsed)`, a gold/amber chip renders inline (`.sl-`

`exceeds-warning`) — deliberately a warning, not a block; the number is always shown, never hidden.

- **Collapsed-by-default reference table** (`renderTierReferenceTable()`) shows all 12 sub-tiers for comparison. Uses a non-standard "collapsed on the very first visit" rule (most collapsible sections in the app show-once-then-collapse; this one pre-seeds its `localStorage` flag so it never auto-expands).
 - **Backend gap:** all calculations are pure client-side functions reading from in-memory state — a real implementation would need a `POST /trades` or similar endpoint to persist what's logged here, since currently a page reload loses the day's logged trades entirely.
-

Trade Manager

The Big Picture (For Normal Users)

Once you're actually in a trade, this tool tracks it for you — not with a live price feed (there isn't one yet), but by you checking in with the current price whenever you want, and the tool telling you where your stop-loss should move to and whether you've hit your target. Type in a strike name like "23200 CE" and it figures out whether you're long or short automatically. As the trade moves in your favor, your stop-loss trails upward to lock in profit, and once it's moved far enough, the tool tells you the trade is risk-free — you genuinely cannot lose money on it anymore, even if it reverses.

The Business Value (For Stakeholders)

This is the product's most behaviorally significant feature, because it's the one most directly aimed at the founding insight: traders don't lose discipline at the planning stage, they lose it while a trade is live and emotions are running. A trailing stop-loss mechanism that's visible and unambiguous (not "I think I should move my stop" but "the tool says move it to exactly here") removes the moment of hesitation where bad decisions get made.

The Blueprint (For Upcoming Developers)

The most heavily iterated feature in the app, with several real bugs caught and fixed during development worth knowing about:

- **Direction inference:** instrument name ending "CE" → Call, "PE" → Put; no suffix (futures/equity) → inferred from which side of entry the stop-loss sits.
- **Trailing SL formula** (`computeTrailingSl(riskPoints, pointsInFavor)`): below +1R, holds the original stop. From +1R onward, $slFromEntry = pointsInFavor - 1.5 \times riskPoints$, continuous (not stepped). `lockedProfit` is non-null once `slFromEntry > 0` — this is what triggers the "risk-free" state.
- **Preview/commit split** (fixed a real bug): typing a price updates the chart live on every keystroke, but target-hit/stopped-out can only be *decided* on Enter or blur. The original implementation evaluated closure on every keystroke, meaning typing "180"

character-by-character could read the partial "1" as a catastrophic adverse move and trigger a false stop-out that then permanently froze, since every closing check was gated behind `(!isClosed)`.

- **Chart axis:** the visual chart uses real PRICE as its vertical axis, not points-in-favor — a short trade's stop-loss (numerically above entry) correctly renders physically higher on screen than its target (numerically below). An earlier version used a "favorable direction = up" abstraction that looked visually backwards for short trades when read against the actual printed price numbers.
 - **Closed-trade outcomes**, each computing realized points/R:R using the actual exit level (the SL's level at close, not a late-typed check-in price that may already be past it): Target hit (green, confetti), Stopped out with profit locked (green, "banked a win" framing), Stopped out with no profit ever locked (neutral framing, not punitive).
 - **Backend gap:** entirely stateless across reloads; a real implementation needs the trade's full check-in history (not just the final outcome) persisted for the Trading Journal and reporting features to draw on later.
-

Broker Connection & Order History Import

The Big Picture (For Normal Users)

If you connect your broker (currently simulated, not a live connection), the app can show you your actual trade history the way your broker shows it — browse by day, month, or financial year, click a day, and pull those trades into your Trading Journal when you're ready. Nothing from your broker counts against your rules until you explicitly import it.

The Business Value (For Stakeholders)

This is the feature that, once connected to a real broker API, eliminates the single biggest reason a discipline tool gets abandoned: manual data entry fatigue. A trader who has to retype every trade by hand stops doing it within a week. Automatic import is what turns this from "a calculator I sometimes use" into "the thing that already knows what I did today."

The Blueprint (For Upcoming Developers)

- **No default-loaded range:** a deliberate fix — the calendar used to auto-populate with 3 months of data on load; now nothing renders until the trader explicitly picks Day/Month/3-Months/Year, including a date-picker for a specific day.
- **Real weekday-aligned calendar** (`(renderBrokerMultiMonthGrid())`): a genuine rebuild after the original implementation just chunked day numbers into rows of 5 with no actual weekday awareness. Now computes `(new Date(year, month, 1).getDay())` to correctly position day 1 under its real weekday, with proper leading blank cells.
- **Order-history table columns** intentionally mirror a real broker's Order History view (Scrip, Qty, Buy Price, Sell Price, Time, Status) rather than a P&L statement — Charges

and Net P&L were deliberately removed from this view after user feedback that this screen's job is "what did I trade," not "how much did I make," with that calculation reserved for after import.

- **Mock execution times** generated within real NSE/BSE market hours (9:15-15:30), chronologically ordered within a day.
 - **Data model gap, explicitly acknowledged in-product:** imported trades currently live in an in-memory array (`tradeHistory`) that resets on reload — confirmed as acceptable for the current prototype stage, but the calendar itself reads from a *separate* mock dataset (`brokerPnlData`) than the imported log, meaning a real implementation needs to deliberately decide whether these stay two different data sources (broker's full history vs. the trader's own curated log) or get unified.
-

Roadmap

The Big Picture (For Normal Users)

This page shows you where you stand on the journey toward your next tier and, further out, toward ₹1 crore — and does the math on how many winning trades it would actually take to get there at your current pace. If your current win rate and average win size mean you're mathematically moving backward, it tells you that plainly, rather than showing an optimistic projection.

The Business Value (For Stakeholders)

The Roadmap is the product's primary motivational/retention surface — it's what gives a trader a reason to open the app on a day they have no trade to log. The deliberately honest framing ("not reachable at this pace") is a trust-building choice: a tool that only ever shows encouraging projections would lose credibility fast with traders who already distrust trading-app marketing.

The Blueprint (For Upcoming Developers)

- **Projection math** uses the gap to the next tier divided by the trader's actual average win (defaulting to a manually entered estimate, intended to be replaced by real logged win/loss data once enough exists).
- **Goal-picker** spans the next tier up through ₹1 crore in fixed increments, with the ₹1 Crore option visually distinguished (gold accent) as the product's namesake milestone.
- Explicitly shows the reverse-engineered required win rate against a stated typical loss size, including the plain-language explanation of why a sub-breakeven win rate makes more trades actively harmful, not neutral.
- **Backend gap:** average-win and win-rate inputs are currently manual; a real implementation should compute these from actual logged Trade Manager/Journal history once that data persists.

Trading Journal

The Big Picture (For Normal Users)

A place to write up the reasoning, emotional state, and outcome of each trade after the fact — not just the numbers, but the thinking behind them, so patterns in your own behavior become visible over time.

The Business Value (For Stakeholders)

Journaling is a well-established discipline-improvement practice in trading psychology literature; including it natively (rather than requiring a separate notebook or spreadsheet) keeps the trader's entire discipline workflow inside one product, which is a meaningful stickiness lever.

The Blueprint (For Upcoming Developers)

Currently a standalone lazy-loaded component (`journal.js`/`journal.html`) with its own card-based entry list and stat summary. Not yet wired to automatically pull from Trade Manager's closed-trade outcomes or imported broker history — a clear next integration point, since manually re-describing a trade that the app already has full data on is exactly the friction this product exists to remove elsewhere.

Suggestions

The Big Picture (For Normal Users)

A curated set of short, plain-language warnings about the behavioral mistakes that quietly cost most new F&O traders money — not trade ideas, but things like "don't take random entries," "holding an option overnight carries real overnight risk," and "expiry-day zero-hero trades are how people blow through their daily limit." Click any one to read the full reasoning behind it.

The Business Value (For Stakeholders)

This is low-cost, high-credibility content that reinforces the product's core positioning (discipline over signals) every time a trader opens it, and serves as a natural landing spot for SEO/educational content marketing once the app has a public-facing presence.

The Blueprint (For Upcoming Developers)

- **Deliberately simple v1 scope, confirmed explicitly during scoping:** a static, always-visible, non-personalized, non-date-aware list. A later version could detect real expiry days and surface the expiry-specific warning only then, or personalize based on a trader's actual logged behavior (e.g., surfacing the revenge-trading warning specifically after a logged loss) — neither is built yet, by design, to keep v1 shippable.

- **List/panel pattern, not cards:** rebuilt from an initial all-cards-always-expanded layout after recognizing it wouldn't scale past a handful of items. Titles render as a compact list ((`FLAT_ITEMS`), a flattened index across all categories); clicking opens a slide-in side panel with Previous/Next navigation that walks the *entire* flat list, not just the current category — confirmed as the more natural reading flow.
 - Content lives as a hardcoded JS array ((`SUGGESTIONS`)) — a real implementation should move this to a CMS-backed endpoint so content can be added without a deploy, especially once the catalog grows toward the "100 per category" scale the product is explicitly designed to handle.
-

Education & Books

The Big Picture (For Normal Users)

Reference material — structured lessons and a curated reading list — for traders who want to go deeper on the concepts the app enforces day to day.

The Business Value (For Stakeholders)

Low-maintenance content surfaces that round out the product as a genuine learning destination, not just a calculator — useful for justifying a subscription price point beyond pure utility.

The Blueprint (For Upcoming Developers)

Static lazy-loaded components, structurally similar to Suggestions pre-rebuild (card-based, fully static). Natural future candidates for the same list/panel scaling pattern if content volume grows.

Strategies

The Big Picture (For Normal Users)

A space for documenting and reviewing specific trading strategies/setups, separate from the day-to-day trade logging in the Journal.

The Business Value (For Stakeholders)

Differentiates casual users from serious ones — a trader actively building out a Strategies library is a strong retention/upsell signal for a future paid tier.

The Blueprint (For Upcoming Developers)

Standalone lazy-loaded component, card-based, no current data dependency on other tabs.

Account

The Big Picture (For Normal Users)

Where you can see your full profile at a glance and change your tier, capital, trading style, or instruments without going through the whole setup wizard from scratch.

The Business Value (For Stakeholders)

Reducing friction to update a profile (rather than forcing a full re-onboard) directly supports data accuracy — a trader whose real capital has changed but whose profile hasn't is a trader the risk engine is silently miscalculating for.

The Blueprint (For Upcoming Developers)

"Edit Profile" routes back into the same onboarding wizard (`showTierSelect()`), which detects existing profile state and lands on a sensible step rather than restarting from Step 1 — confirmed behavior, not an oversight. Avatar uses initials generated from the trader's name; gradient styling matches the header avatar for visual consistency.

Subscription Pricing

The Big Picture (For Normal Users)

Shows the available plans, with your current tier's matching plan visually highlighted so it's clear which one applies to you.

The Business Value (For Stakeholders)

The actual monetization surface — currently a static pricing display with no payment integration. The "popular plan" gold-highlighted treatment is a standard conversion-optimization pattern carried over from the visual rebrand.

The Blueprint (For Upcoming Developers)

Static `pricing.html`/no dedicated JS beyond tier-highlight logic shared with the rest of the app. **No payment gateway integrated** — this is the most significant missing piece between the current prototype and an actual revenue-generating product.

Visual Theming (Light/Dark Mode)

The Big Picture (For Normal Users)

A sun/moon toggle in the header switches the entire app between a light theme and a dark theme, and remembers your choice the next time you open the app.

The Business Value (For Stakeholders)

A small but expected feature for a modern trading tool — traders frequently have the app open for long stretches during market hours, and dark mode is a meaningful comfort/accessibility feature for that use case, with a low implementation cost relative to its perceived polish.

The Blueprint (For Upcoming Developers)

- Implemented as a fully separate stylesheet (`dark-theme.css`), loaded after the base `dashboard.css`, with every rule scoped under `body.dark-mode` — a deliberate architectural choice to avoid rewriting the entire existing stylesheet into CSS custom properties, keeping the (much larger, already-stable) light theme completely unmodified and unrisks.
 - Toggle (`toggleDarkMode()`) adds/removes the class and persists the choice to `localStorage`; `applyDarkModePreference()` runs at the very start of `DOMContentLoaded` to avoid a flash of light mode on load.
 - **Coverage:** the whole application, sidebar included, confirmed explicitly in scope (the sidebar's "always dark" base theme is intentionally given a distinct, deeper dark-mode treatment rather than left unchanged, so toggling is visibly meaningful everywhere).
 - **Verification approach worth noting for future contributors:** every class referenced in `dark-theme.css` was cross-checked against actual selectors in `dashboard.css` before shipping, since a CSS selector that doesn't match anything fails silently with no error — this caught 3 stale/incorrect references during development that would otherwise have shipped as quietly broken dark-mode coverage on those specific elements.
 - **Not yet covered:** the public marketing website (`website/index.html`) — confirmed explicitly as a deliberate scope boundary, app-first.
-

Appendix: Known Platform-Level Gaps

For developers picking this up next, these apply across most features above rather than to any single one:

- **No backend / no persistence.** Every screen's state lives in JS closures in the browser tab. A full page reload loses profile setup mid-wizard, today's logged trades, any open Trade Manager position, and imported broker history. This is the highest-priority architectural gap.
- **No real broker integration.** All broker connections, balances, and order histories are locally generated mock data (`generateMockBrokerPnlHistory()` and similar). Building a real broker API integration (Zerodha Kite Connect or similar) is the natural next major milestone.
- **No payment processing,** despite a fully designed pricing page.
- **No authentication backend** — `auth.js` is an explicitly-labeled mock session layer.

